
Protocolos de comunicación

De los cables a la red: UART, I2C, SPI, JTAG, SSH y MQTT

EL5841 Taller de Sistemas Embebidos · ITCR · II Semestre 2026

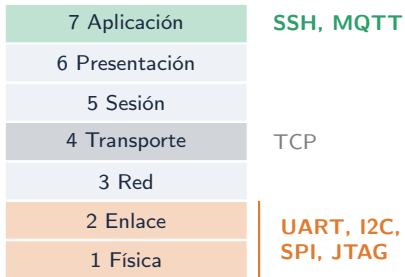
Luis Diego García Rojas

¿Qué es un protocolo?

Un **acuerdo** entre dispositivos sobre:

- cómo se representan los datos,
- quién habla y cuándo,
- saber cuando el mensaje llegó.

Sin acuerdo, los bits llegan pero **nadie** los entiende.



Bajo nivel: voltajes y bits. Alto nivel: viaja sobre TCP/IP.

Cinco preguntas para cada protocolo

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

1. ¿Qué problema resuelve?
2. ¿Cómo funciona?
3. ¿Cómo se ve?
4. ¿Cuándo lo usaría?
5. ¿Qué gano y qué pierdo?

Ningún protocolo es el mejor.

Cada uno resuelve **un** problema y paga un precio por hacerlo.

Para recordar

Síncrono: comparten una línea de reloj.

Asíncrono: sin reloj; acuerdan la velocidad antes.

Full duplex: ambos pueden hablar a la vez.

Half duplex: hablan por turnos.

Bajo nivel

Dentro de la placa: UART, I2C, SPI y JTAG

UART

Serie

Asíncrono

Full duplex

Punto a punto

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

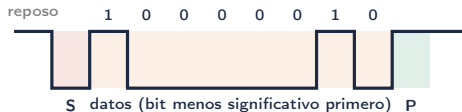
¿Qué problema resuelve?

Dos dispositivos intercambian datos con el mínimo de cables y **sin reloj compartido**.

¿Cómo funciona?

- TX de uno al RX del otro (**cruzados**) y GND común.
- Ambos acuerdan antes la velocidad (**baudios**).
- Cada byte va enmarcado: bit de inicio, 8 datos, bit de parada.

¿Cómo se ve?



La letra A = $0x41 = 0100\ 0001$. En reposo la línea queda en alto; el bit de inicio la baja y avisa "viene un byte".

UART

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

¿Cuándo lo usaría?

Comunicar dos placas (el Lector y la Puerta del taller), la consola de depuración (`Serial.println`), módulos GPS, Bluetooth o módems.

¿Qué gano?

- Solo 2 cables (más GND).
- Simple y presente en casi todo micro.
- Ambos pueden hablar a la vez (*full duplex*).

¿Qué pierdo?

- Solo dos dispositivos.
- Si los baudios no coinciden, llega basura.
- No confirma que el dato llegó.

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

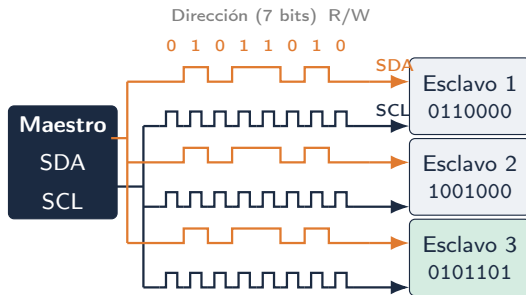
¿Qué problema resuelve?

Conectar **muchos** sensores a un micro sin gastar pines por cada uno.

¿Cómo funciona?

- Dos líneas compartidas: SDA (datos) y SCL (reloj del maestro).
- Cada esclavo tiene una **dirección** de 7 bits.
- El maestro llama por dirección; quien recibe confirma con **ACK**.

¿Cómo se ve?



Todos ven la dirección 0101101; solo el esclavo 3 responde (R/W = 0: escritura).

I2C

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

¿Cuándo lo usaría?

Sensores de temperatura o movimiento, memorias EEPROM, relojes de tiempo real y pantallas LCD con módulo PCF8574.

¿Qué gano?

- 2 cables para decenas de dispositivos.
- Un ACK confirma cada byte.
- Agregar un sensor no cuesta pines (comparten los mismos cables).

¿Qué pierdo?

- Lento: 100–400 kHz típicos.
- Solo distancias cortas, dentro de la placa.
- Dos chips iguales chocan en la misma dirección.

SPI

Serie

Síncrono

Full duplex

Maestro-esclavo

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

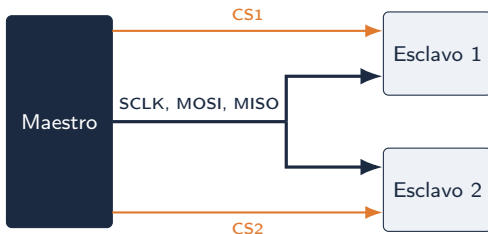
¿Qué problema resuelve?

Mover **muchos datos rápido** entre el micro y un periférico, como una pantalla o una tarjeta SD.

¿Cómo funciona?

- El maestro genera el reloj (SCLK).
- MOSI y MISO envían y reciben **al mismo tiempo**.
- Una línea CS por esclavo: en bajo, “te hablo a vos”.

¿Cómo se ve?



Tres líneas compartidas y una de selección por cada esclavo.

SPI

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

¿Cuándo lo usaría?

Tarjetas microSD, pantallas a color, memorias flash y convertidores analógico-digitales rápidos.

¿Qué gano?

- Muy rápido: decenas de MHz.
- Envía y recibe a la vez.
- Sin direcciones: más simple que I2C.

¿Qué pierdo?

- Más pines: 3 más uno por esclavo.
- No confirma la recepción.
- Sin estándar formal: 4 modos que hay que configurar.

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

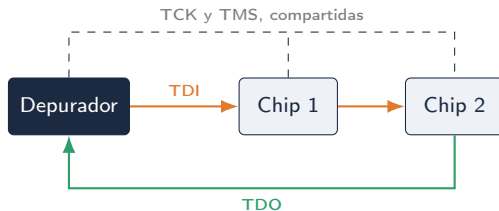
¿Qué problema resuelve?

Programar y **depurar** un chip ya soldado, o comprobar que las soldaduras de la placa estén bien.

¿Cómo funciona?

- Estándar IEEE 1149.1.
- TCK (reloj) y TMS (modo) controlan una máquina de estados en el chip.
- TDI entra y TDO sale: varios chips forman una **cadena**.

¿Cómo se ve?



Los datos recorren todos los chips en fila y vuelven al depurador.

JTAG

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

¿Cuándo lo usaría?

Grabar el firmware de un micro o una FPGA, depurar paso a paso con puntos de interrupción (OpenOCD y GDB) y probar placas en la fábrica.

¿Qué gano?

- Control total: registros, memoria, ejecución.
- Depurar sin tocar el código.
- Probar la placa sin sondas.

¿Qué pierdo?

- No sirve para comunicar datos en el uso normal.
- 4–5 pines (en ARM, SWD usa solo 2).
- Un puerto JTAG abierto en un producto es una **puerta trasera**.

Alto nivel

A través de la red: SSH y MQTT, sobre TCP/IP

SSH

Capa de aplicación

Sobre TCP

Full duplex

Cliente-servidor

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

¿Qué problema resuelve?

Administrar una máquina remota por una red insegura sin que nadie lea ni falsifique lo que se envía. Su antecesor, Telnet, mandaba todo en texto plano.

¿Cómo funciona?

- Cliente y servidor sobre TCP (puerto 22).
- El servidor prueba su identidad: la **huella**.
- El usuario se autentica con contraseña o llave.

¿Cómo se ve?

```
$ ssh -p 2222 tecnico@localhost
ED25519 key fingerprint is SHA256:3A1d...
Are you sure you want to continue? yes
tecnico@localhost's password:
tecnico@servidor-lab:~$ puerta abrir
Puerta ABIERTA.
```

Todo lo que viaja entre las dos máquinas va cifrado.

SSH

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

¿Cuándo lo usaría?

Administrar una Raspberry Pi sin monitor, entrar a servidores, copiar archivos de forma segura (scp, sftp). Es la misión 3 del taller.

¿Qué gano?

- Todo cifrado.
- El servidor demuestra quién es.
- Llaves en vez de contraseñas.

¿Qué pierdo?

- Necesita red y sistema operativo: no corre en un Arduino Uno.
- Es tan seguro como su uso: contraseñas débiles o ignorar la alerta de huella lo rompen.

MQTT

Capa de aplicación

Sobre TCP

Publicar/suscribir

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

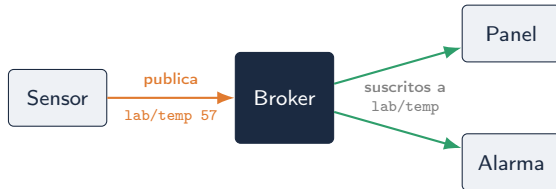
¿Qué problema resuelve?

Muchos dispositivos IoT envían datos a muchos interesados, con redes inestables y poco ancho de banda.

¿Cómo funciona?

- **Publicar / suscribir**, con un **broker** en el centro.
- Se publica en un tema, como lab/temp.
- Solo recibe quien se suscribió a ese tema. Nadie conoce a nadie.

¿Cómo se ve?



El sensor no sabe quién lo escucha: solo le habla al broker.

MQTT

Problema

¿Cómo funciona?

¿Cómo se ve?

¿Cuándo?

Gano / pierdo

¿Cuándo lo usaría?

Sensores con ESP32 que reportan a la nube, domótica y telemetría de muchos nodos.

¿Qué gano?

- Mensajes livianos: encabezado mínimo de 2 bytes.
- Agregar receptores no cambia al emisor.
- Calidad de servicio (QoS) para elegir la garantía de entrega.

¿Qué pierdo?

- Si el broker cae, cae todo.
- Necesita red TCP/IP.
- Sin cifrado por defecto: hay que activar TLS.

Comparación

	Reloj	Cables	Quién con quién	Velocidad	Alcance
UART	Asíncrono	2 + GND	Punto a punto	kbps	Corto
I2C	Síncrono	2	Un maestro, muchos esclavos	100–400 kHz	En la placa
SPI	Síncrono	3 + 1 por esclavo	Un maestro, pocos esclavos	Decenas de MHz	En la placa
JTAG	Síncrono	4–5	Depurador y cadena de chips	MHz	En la placa
SSH	—	Red TCP/IP	Cliente y servidor	La de la red	Internet
MQTT	—	Red TCP/IP	Muchos a muchos (broker)	La de la red	Internet

¿Cuál elijo?

Dos placas que necesitan hablar entre sí

→ UART

Muchos sensores y pocos pines libres

→ I2C

Una pantalla o una SD: necesito velocidad

→ SPI

Grabar o depurar el chip

→ JTAG

Administrar una máquina remota con seguridad

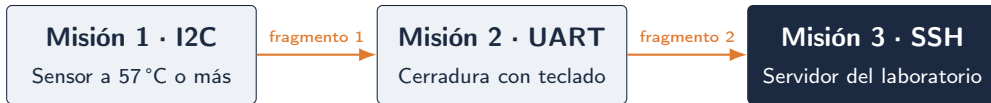
→ SSH

Muchos dispositivos IoT reportando datos

→ MQTT

Laboratorio cerrado

Un corte de luz dejó el laboratorio en **modo seguro**. Andrea dividió la contraseña del servidor en **dos fragmentos**, cada uno en un dispositivo que habla un protocolo distinto: **control dual**.



Primero: la demo de I2C.

Referencias

- H. Zimmermann, "OSI Reference Model—The ISO Model of Architecture for Open Systems Interconnection", *IEEE Trans. on Communications*, vol. 28, no. 4, pp. 425–432, 1980.
- NXP Semiconductors, *UM10204: I2C-bus specification and user manual*, Rev. 7.0, 2021.
- IEEE, *Std 1149.1: Standard for Test Access Port and Boundary-Scan Architecture*.
- T. Ylonen y C. Lonvick, *RFC 4251: The Secure Shell (SSH) Protocol Architecture*, IETF, 2006.
- OASIS, *MQTT Version 5.0*, OASIS Standard, 2019.
- Microchip Technology, *ATmega328P Datasheet*: secciones de USART, SPI y TWI.
- Analog Devices, *TMP35/TMP36/TMP37 Low Voltage Temperature Sensors*, hoja de datos.
- Arduino, referencia de las librerías `Wire` y `SoftwareSerial`, [arduino.cc](https://www.arduino.cc).

¿Preguntas?

Ahora, a abrir el laboratorio.